Project 8.1 — OpsPilot-On-Call-Assistant-Over-Discovered-MCP-Tools

A practice twin of Project 8 — MCPBridge. Same skills — an agent that discovers its tools at runtime from live MCP servers, runs a ReAct loop over them, connects to several servers at once with namespacing, and degrades gracefully when one is down — on a **new, real-world job**. **There are no steps here on purpose.** Read the story, build the whole thing yourself, and only open the hints or the solution if you get stuck.

Type: Single agent (ReAct over discovered MCP tools) · **Stack:** Python, `mcp` SDK, OpenAI SDK (function calling), asyncio, Pydantic · **Level:** Intermediate–Advanced

> **Stuck, or want to check your work?** Full hints · Complete solution

The Story — what you are going to build

It's 2pm and checkout is throwing 503s. The on-call engineer has three browser tabs open: the service-status dashboard, the deploy history, and a wiki full of runbooks. They're doing the same thing a human always does in an incident — *gather from a few different places, then decide* — and every second of tab-switching is a second the site is down.

You are going to build **OpsPilot**: an on-call assistant that does the gathering. You ask it one plain question — *"Is checkout down, and what does the runbook say to do?"* — and it answers by reaching into the **same tools the engineer would**, except those tools don't live in OpsPilot's code. They live on **separate internal services**, each exposing its tools over MCP:

- A **status service** — `get_service_status(service)`, `recent_deploys(service)`.
- A **runbooks service** — `search_runbooks(query)`, `read_runbook(runbook_id)`.

OpsPilot connects to both at startup, asks each *"what tools do you have?"*, and runs a ReAct loop over whatever it's handed. It never hardcodes a single tool — because at a real company the status service and the runbook wiki are owned by **different teams**, change on their own schedules, and the last thing you want is OpsPilot carrying a stale copy of either.

The real reason this shape matters: the status team adds a new `get_error_budget` tool next month. With OpsPilot, it just *appears* — the agent discovers it on the next restart, no code change. A hardcoded agent would silently never know it existed.

How it works, from the moment you ask (picture the whole machine)

Read this once and the whole project should form in your head.

OpsPilot starts and connects to every configured MCP service over stdio, doing the MCP handshake with each. For each one it calls `list_tools()` and gets back that service's tools — names, descriptions, input schemas. It gives every tool a **namespaced** name (`status__get_service_status`, `runbooks__search_runbooks`) and remembers which real service each name belongs to, in a routing table. Why namespace? Because two services could both expose a `search` tool, and the model needs one unambiguous name per tool — and because the function-name field only allows letters, digits, `_` and `-`, so the separator is `__`, never `.`.

You ask your question. OpsPilot hands the model your question **plus the discovered tool list** (converted into the shape the model's function-calling expects — an MCP tool's input schema is already JSON Schema, so that's mostly a copy). The model picks a tool — say `status__get_service_status` with `{"service": "checkout"}`. OpsPilot looks that namespaced name up in its routing table, dispatches the call to the **status** service, gets the result back, and feeds it to the model as the observation. The model picks the next tool — maybe `runbooks__search_runbooks`, then `runbooks__read_runbook` with an id from the first result — until it has enough to answer. A hard step cap stops it from looping forever.

And when the status service is **down** (which, during an incident, is exactly when it might be): OpsPilot's connection to it times out, gets logged and skipped, and the agent runs on the **runbooks service alone** — answering what it can and saying plainly it couldn't reach live status, instead of hanging or crashing. That resilience is the difference between a demo and something you'd actually trust on-call.

That's the entire machine: **connect → discover → namespace → ask the model → route each call to the right service → observe → repeat, and never let one dead service take the whole thing down.**

What you are building (the pieces)

- A **server pool** that connects to every configured MCP service, discovers its tools, namespaces them, and routes each call back to the right service.
- A **schema bridge** that turns a discovered MCP tool into an OpenAI function schema, and pulls the text out of a tool result.
- A **hand-built ReAct loop** (no `create_agent` / `AgentExecutor`) that runs over the discovered tools, bounded by a hard step limit.
- **Resilience:** a per-service connect timeout (a service that won't start is skipped, not fatal), and a try/except around each call (a mid-incident drop becomes a clear observation, not a crash).
- Two small **example MCP services** (status, runbooks) so the whole thing runs with no external setup.

Inputs: a plain on-call question. **Outputs:** an answer assembled from discovered tools, plus a visible log of which service each call hit. **Rules:** no tool is written into the agent's own code; two services run at once with correct routing; one dead service degrades the agent instead of killing it.

A real example to build and test against

- *"Is checkout down, and what does the runbook say to do?"* → `status__get_service_status(checkout)` → `runbooks__search_runbooks("checkout")` → `runbooks__read_runbook("rb-checkout")` → an answer that combines live status with the runbook steps. **Two services, one question.**
- *"Is search slow right now?"* → just `status__get_service_status(search)` — one service, one call.
- The same incident question **with the status service not started** → OpsPilot connects to runbooks only, answers from the runbook, and says clearly it couldn't reach live status.

How to practice this

1. Read this story. **Don't open the solution yet.**
2. Plan it on paper: the pool (connect + discover + namespace + route), the bridge, the loop, and what happens when a service is down.
3. Build it yourself. A good order: one service discovered and printed → a ReAct loop over it → add the second service and prove routing → add the timeout/skip and the mid-call catch. But it's your call.
4. Test it on the real example: the two-service incident question, the one-service question, and the status-service-down case.
5. **Only if stuck**, read the hints. **To compare**, read the complete solution.

Why this project is useful (read this at the end)

This is the real shape of a lot of production agents: the agent and the tools it needs are built and run by **different teams**, and the only contract between them is a protocol. An on-call copilot, an internal "ask-anything" assistant, a customer-support agent that reaches billing + orders + shipping services — they're all this: connect to the services you're allowed to use, discover what they offer *right now*, and route each call to the right one, surviving the ones that are down. If you can build OpsPilot from this story alone, you can build any of them — and you'll have internalized the habit that separates a toy agent from a production one: **discover your tools, don't hardcode them, and never let one dead dependency take the whole agent with it.**

Done when

- OpsPilot's tool list comes entirely from `list_tools()` at runtime — no tool written into its own code.
- The incident question routes to both services; the one-service question uses one; the logs show each call hit the service you expected.
- With the status service down, OpsPilot connects to runbooks only, answers what it can, and says it couldn't reach live status — no hang, no crash.
- Your tests pass: the settings, the schema bridge (no network), and a live run of the agent against the real example services.

The full build, every file, is in the complete solution — use it to check your work, not to skip the thinking.